

Advanced Image Processing & GUI Development

Assignment 4

Deadline: 23rd July

Instructions

- There are **8 questions** in this assignment.
 - **Questions 1–5 are compulsory.**
 - **Questions 6–8 are optional challenge questions**, carrying **Q6 – 1 Point, Q7 – 2 Points, and Q8 – 4 Points** respectively.
 - **Minimum points requirement:** You must score at least **N points** overall (N will be announced separately on the whatsapp group). Every previous assignment you did not submit costs you **2 points**, and every class you missed costs you **1 point**, deducted from this running total. Optional questions (Q6–Q8) are the primary way to make up for these deductions.
 - **Build on your final project, don't restart.** Take the final, working version of your **CubeCanvas** code and add the features asked for in each question directly into it. You are not expected to write 8 separate disconnected scripts – one evolving project is fine and preferred.
 - **Poll for optional questions:** A poll will be shared on the group – tick which of Q6/Q7/Q8 you intend to attempt so we know what to expect from you.
 - Write clean, readable and modular code, and use comments wherever necessary.
 - Organize your solution locally before uploading it to GitHub.
-

Submission Guidelines

Repository

https://github.com/SHUB-1806/CubeCanvas_Speedcubing

Your submission must be placed inside

`Assignments/Assignment4/<Your_Name>/`

Unlike previous assignments, you do **not** need a separate folder per question this time. Since everything is being built into one evolving project, submit a **single folder** containing your final project code (all relevant `.py` files and any supporting assets), like so:

```
<Your_Name>/
|
|-- main.py
|-- image_widgets.py
|-- image_processing.py
|-- pdf_export.py
|-- ... (any other files your project needs)
\-- demo_video.mp4
```

Record a **single** screen recording that runs through **all** the features you implemented for this assignment, one after another, in one continuous take. Do not record a separate video per question.

Git Workflow

Follow the steps below to submit your assignment.

1. Clone the repository

```
git clone https://github.com/SHUB-1806/CubeCanvas_Speedcubing.git
cd CubeCanvas_Speedcubing
```

2. Create your submission directory

```
Assignments/
  Assignment4/
    <Your_Name>/
```

3. Copy your complete project folder (code + demo video) into this directory.

4. Verify your changes

```
git status
```

5. Stage, commit and push

```
git add Assignments/Assignment4/<Your_Name>
```

```
git commit -m "Assignment 4 Submission - <Your_Name>"
```

```
git push origin main
```

6. Open GitHub in your browser and create a Pull Request.

Use the following format:

Name -- Roll Number -- Assignment 4 Submission

Example:

Shubham Joshi -- 240500 -- Assignment 4 Submission

Unable to Use Git?

If you are unable to use Git, create the same folder structure directly on GitHub under

Assignments/Assignment4/

and upload your project folder to it. Avoid uploading all files directly inside the Assignment4 folder.

Unable to do anything?

DM me

Question 1: RGB Histogram Viewer (Easy)

Add a function

```
show_histogram(image)
```

to your project that displays the Red, Green, and Blue histograms of any uploaded image – the frequency of each pixel intensity (0–255) per channel, plotted together (e.g. with Matplotlib) so the distributions can be compared at a glance. Wire it up to a button in your GUI so it can be triggered on the currently loaded/edited image.

Test it on at least a bright image, a dark image, and your own colorful mosaic output, and make sure the plot updates correctly for each.

you may append this feature in the final code of our project or you may just demonstrate this in simple code written from scratch, its upto you.

Question 2: Add New Image Effects (Easy–Medium)

Your project currently supports brightness, contrast, saturation, sharpness, blur, and flipping. Implement **any four** additional effects and integrate them into your existing effects panel so the user can toggle/apply them interactively and see the result update live:

- Negative (Invert)
 - White balance correction
 - Gamma correction
-

Question 3: Improve the Pixelation Algorithm (Medium)

Your project currently reduces an image to a limited palette using nearest-color matching and Floyd–Steinberg dithering. Implement **any two** additional pixelation/quantization methods and add a way to switch between **all** available algorithms (existing + new) from the GUI, so outputs can be compared directly:

- Ordered (Bayer Matrix) Dithering
- Gradient-Based Quantization
- Atkinson Dithering

Run all implemented algorithms on at least three different images and note (briefly, as a comment or short write-up) how their visual quality differs

Question 4: Undo / Redo System (Medium–Hard)

Replace the single “Revert Changes” button in your project with a proper Undo/Redo system:

1. Every time the user applies an effect, slider change, flip, or quantization, push the resulting image state onto an undo **stack** (read upon this topic, will be very useful later too in DS Algo.)
2. An “Undo” button restores the previous state and moves the current one to a redo stack.
3. A “Redo” button re-applies the most recently undone state.
4. Performing any new edit after an Undo clears the redo stack.
5. There should be no fixed limit on how many steps can be undone/redone.

Question 5: Multi Cube Size Support (Hard)

Currently, **CubeCanvas** assumes every Rubik's Cube is a **3×3** cube while generating the PDF build guide. Add a **dropdown menu** beside the **Export** button that allows the user to choose the cube type before exporting. The available options should be:

- 3×3
- 4×4
- 5×5

Based on the selected cube type, the PDF generation pipeline should automatically adapt the chunk size. Consequently, the following should update correctly:

- Chunk partitioning of the mosaic.
- The number of stickers displayed in each chunk.
- Grid overlays in the PDF.
- Cube boundaries within each chunk.
- Chunk labels and page layouts.

Verify that the exported PDF correctly represents the selected cube size for the same input image without any distortion or incorrect chunking. (show the output pdf of 5x5 selected in the video submission)

Optional Challenge Questions (Q6–Q8)

Tick which of these you intend to attempt on the poll shared on the group.

Question 6: Interactive Color Picker (Optional – 1 Point)

Currently, cube palette colors are entered manually as hex text. Replace or augment this with a clickable color picker:

1. Clicking a color swatch box opens a native color picker dialog (Python’s standard library provides one out of the box: `from tkinter.colorchooser import askcolor`).
 2. Choosing a color updates the swatch, the hex entry, and immediately regenerates the mosaic with the new color.
 3. Cancelling the dialog leaves the existing color unchanged.
-

Question 7: Cube Sticker Color Extractor (Optional – 2 Points)

Allow the user to upload a straight-on photo of one real Rubik’s Cube face, and:

1. Detect the 3×3 sticker grid on the face.
2. Extract the average color of each of the 9 stickers and report each as a hex code.
3. Let the user assign each detected sticker to one of the six official palette colors.
4. Update your project’s active color palette with these assignments.

Hint: Computer-vision libraries like OpenCV (cv2) are useful here for detecting the grid and computing region averages; working in HSV rather than RGB can also make separating the colors easier. (ask **Rubibot mentees** for help if you need, they have done similar kind of work in their project)

Question 8: Algorithm Comparison Dashboard (Optional – 4 Points)

Build a dashboard that:

1. Runs **every** pixelation algorithm your project implements (nearest-color, Floyd–Steinberg, and your Q3 additions) on the same source image and palette.
 2. Displays every algorithm’s output side-by-side in a grid, each clearly labeled with its algorithm name.
 3. Lets the user click on any output to select it as the “final” mosaic.
 4. Exports only the currently selected mosaic to PDF, reusing your existing PDF export pipeline.
 5. **Bonus:** shows the execution time of each algorithm next to its label.
-

THANK YOU !!